take a block that receives an account value. Now, we can get the actual account balance and put it into the failure message.

We'll need to keep the first part of the failure message that's provided by RSpec—the part that reads, expected ... to have a balance of 35. By calling super(), you delegate to the existing RSpec implementation for this part.

Matchers Are Just Classes

The reason that matchers can call super() just like Ruby classes is that they *are* Ruby classes. RSpec::Matchers.define creates a Ruby class, and the block you pass to it is the class body. Inside the block, we can do anything we'd do inside a Ruby class, including defining private helper methods.

Most of the failure message will be the same for regular and negated matchers, so we've abstracted that common part into a failure_reason helper method. Take a look at the output with these changes in place:

```
1) `have_a_balance_of(amount)` fails when the balance does not match
   Failure/Error: expect(account).to have_a_balance_of(35)
     expected #<Account name="Checking"> to have a balance of 35, but had a ↩
     balance of 30
   # ./spec/initial_account_spec.rb:17:in `block (2 levels) in <top
```

Much more helpful.

Adding a Fluent Interface

Many of RSpec's built-in matchers use a *fluent interface*,[4] where you can chain one method after another to create an easy-to-understand expectation:

- be_within(0.1).of(50)
- change { ... }.from(x).to(y)
- output(/warning/).to_stderr

It's easy to add these same kind of fluent interfaces to your own custom matchers. Continuing the account balance example, suppose our Account class provides two methods for querying the balance:

- current_balance
- balance_as_of(date)

It'd be nice to support both of these methods from the same matcher, by allowing specs to tack on an optional as_of modifier with a date:

4. https://martinfowler.com/bliki/FluentInterface.html